Funciones

Las funciones de PostgreSQL, también conocidas como Procedimientos almacenados (Store Procedures), como en todos los lenguajes de programación, tienen por objetivo encapsular cierta lógica de forma tal que podamos obtener algún resultado a partir de su invocación. Al igual que en un lenguaje de programación procedural, éstas funciones pueden recibir parámetros y devolver un único valor como resultado. Respecto de esto último, cabe mencionar que los valores o bien pueden ser de tipos de datos nativos como enteros o cadenas o bien pueden ser de tipo registro; este último permite que una función devuelva, tal como mencionamos en otras unidades, una tabla.

Las funciones permiten también la reutilización de la base de datos ya que otras aplicaciones pueden interactuar directamente con ellas en lugar de hacerlo a través de código u otras consultas, garantizando de esta forma un comportamiento homogéneo.

PostgreSQL tiene la particularidad de que las funciones se pueden desarrollar utilizando lenguaje SQL, PL / pgSQL, C, Python, etc.

Sintaxis

```
CREATE OR REPLACE FUNCTION <nombre>([[<mode>] <arg1> <type>, ...,[<mode>]
<argN> <type>]) RETURNS <ret_type> AS $$
DECLARE <var_name> <var_type>
BEGIN
...;
END;
$$ LANGUAGE <language>;
```

mode: IN|OUT|INOUT

type: tipo de cada argumento. Si el tipo está basado en alguna columna, se puede utilizar la siguiente expresión: <table_name>.<column_name>%TYPE

ret_type: tipo de retorno de la función. Puede ser un tipo básico o TABLE, en este caso hay que especificar las columnas y sus tipos.

language: puede ser sql; plpgsql; plpythonu

Tipos de funciones

Los tipos de funciones están determinados principalmente por el lenguaje que se utilice para programarlas.

En el caso de ***SQL*** sólo podrán efectuarse operaciones especificadas en dicho lenguaje. Si requerimos especificar una lógica más compleja, podemos utilizar ***plpgsql*** que nos habilita el uso de estructuras de decisión, bucles, etc.

También existe el tipo ***plpythonu*** que nos permite escribir las funciones en python, con todo su potencial.

Veamos algunos ejemplos

Utilizando lenguaje sql:

```
-- Ejemplo de función que permite actualizar un registro de una tabla
CREATE FUNCTION tf1 (accountno integer, debit numeric) RETURNS integer AS $$
  UPDATE bank
    SET balance = balance - debit
    WHERE accountno = tf1.accountno;
  SELECT 1;
$$ LANGUAGE SQL;


-- Ejemplo de función cuyo tipo de retorno es una tabla con 2 campos
CREATE FUNCTION dup(int) RETURNS TABLE(f1 int, f2 text) AS $$
  SELECT $1, CAST($1 AS text) || ' is text'
$$ LANGUAGE SQL;


SELECT * FROM dup(42);

-- Genera la siguiente salida
 f1 |     f2
----+------------
 42 | 42 is text
(1 row)
```

Cabe destacar que el segundo ejemplo, la función dup podría ser utilizada en la cláusula from de alguna otra consulta ya que devuelve una tabla.

Utilizando lenguaje plpgsql:

En caso de necesitar implementar cierta lógica que involucre toma de decisiones, iteraciones, etc, debemos desarrollar la función utilizando lenguaje plpgsql.

```
-- Ejemplo de función que valida un par de credenciales
CREATE OR REPLACE FUNCTION check_password(uname TEXT, pass TEXT) RETURNS BOOLEAN AS
$$
DECLARE passed BOOLEAN;
BEGIN
  passed := false;
  SELECT (password = $2) INTO passed
  FROM cliente
  WHERE name = $1;

  -- en caso de que el query no retorne filas, la variable passed contiene NULL.
  IF passed IS NULL THEN
      passed := false;
  END IF;

  RETURN passed;
END;
$$ LANGUAGE plpgsql;


-- Ejemplo de función que suma 2 enteros si y sólo si ambos son positivos
CREATE FUNCTION sum_positive(n1 integer, n2 integer) RETURNS integer AS $$
DECLARE sum integer;
BEGIN
  IF(n1 > 0 AND n2 > 0) THEN
    RETURN n1+n2;
  ELSE
    RAISE NOTICE ‘Error!!’;
  END IF;
END;
$$ LANGUAGE plpgsql;


-- Ejemplo de función que devuelve el registro con el id mayor.
CREATE OR REPLACE FUNCTION last_record() RETURNS provincias AS $$
DECLARE  t2_row provincias%ROWTYPE;
BEGIN
       select * into t2_row
       from provincias order by prv_id desc limit 1;

       return t2_row;
END;

$$ LANGUAGE plpgsql;
```

Es interesante analizar el segundo ejemplo, la función *sum_positive*, ya que como se puede ver utiliza la instrucción `RAISE` para lanzar una excepción en caso de que alguna validación no se cumpla o se quiera/necesite informar de algún error producido dentro de la función.

Para más información sobre el lenguaje plpgsql ver:

- https://www.postgresql.org/docs/current/static/plpgsql.html

Trabajando con excepciones

Como vimos en la sección anterior, si desde una función debemos lanzar una excepción, podemos utilizar la instrucción *RAISE* para realizarlo.

La sintaxis de la instrucción RAISE es la siguiente:

```
RAISE [nivel] [mensaje]
```

nivel: permite indicar el nivel de error. Puede ser uno de DEBUG, LOG, INFO, NOTICE, WARNING, o EXCEPTION. El nivel EXCEPTION es la opción por defecto y es el único de todos los niveles que permite cortar la ejecución de la función.

mensaje: permite generar un mensaje para describir el error.

Cabe destacar que en el archivo *postgresql.conf* podemos editar el nivel de los mensajes que deseamos registrar. Se puede configurar las variables *log_min_messages* y *client_min_messages* para configurar el nivel de los mensajes a reportar en el archivo de log y al cliente respectivamente.

Retomando el ejemplo anterior, tenemos:

```
-- Ejemplo de función que valida un par de credenciales
CREATE OR REPLACE FUNCTION check_password(uname TEXT, pass TEXT) RETURNS BOOLEAN AS
$$
DECLARE passed BOOLEAN;
BEGIN
  passed := false;
  SELECT (password = $2) INTO passed
  FROM cliente
  WHERE name = $1;

  -- en caso de que el query no retorne filas, la variable passed contiene NULL.
  IF passed IS NULL THEN
```

```
    RAISE LOG ‘El usuario % no existe’, uname;
       passed := false;
  END IF;

  RETURN passed;
END;
$$ LANGUAGE plpgsql;


-- Al ejecutar la función
select check_password('juan', 'clave_1234');

-- Obtenemos la salida
 check_password
----------------
 f
(1 row)


-- Y si vemos el archivo de log, veremos el siguiente mensaje
2021-10-28 03:45:13.093 UTC [49] LOG:  El usuario juan no existe
2021-10-28 03:45:13.093 UTC [49] CONTEXT:  PL/pgSQL function
check_password(text,text) line 10 at RAISE
2021-10-28 03:45:13.093 UTC [49] STATEMENT:  select check_password('juan',
'clave_1234');
```

Para más información sobre el manejo de errores ver:

- https://www.postgresql.org/docs/14/plpgsql-errors-and-messages.htm
- https://www.postgresql.org/docs/14/errcodes-appendix.html

Triggers

Básicamente es una indicación que podemos proporcionar al motor de base de datos para que ejecute una función ante determinado evento. La función que ejecute puede ser de cualquier tipo (sql, plpsql, plpythonu, etc) y debe haber sido creada previamente a la definición del trigger. La única salvedad es que la función no debe definir parámetros y su tipo de retorno debe ser trigger.

Tipos y Usos

PostgreSQL soporta triggers sobre tablas y vistas.

Sobre las tablas los tipos de triggers soportados son:

- BEFORE: se ejecuta antes que el evento para el cual se definió
- AFTER : se ejecuta inmediatamente después que el evento para el cual se definió

Los eventos sobre los cuales se pueden disparar son:

- INSERT
- UPDATE
- DELETE
- TRUNCATE

Los triggers pueden ser definidos para que ejecuten, la función que tienen definida, por cada fila o una sola vez por la instrucción invocada.

Algo a tener en cuenta es que en triggers sobre eventos de UPDATE se puede especificar que el trigger se dispare sí y sólo sí determinadas columnas son afectadas por dicha operación.

When	Event	Row-level	Statement-level
Before	insert/update/delete	Tables	Tables + Views
	truncate	-	Tables
After	insert/update/delete	Tables	Tables + Views
	truncate	-	Tables
Instead Of	insert/update/delete	Views	-
	truncate	-	-

Registros especiales

Como vimos, en la definición de la función que es ejecutada por un trigger, no debe recibir parámetros. Ahora bien, el motor de base de datos inyecta a estas funciones, de forma automática, algunos datos (registros) que dependen del evento.

Registro NEW

Este registro contiene la información de los datos a insertar o actualizar en la tabla en cuestión. Este registro es inyectado en la operaciones de insert o update sólo para triggers que se ejecutan a nivel de filas.

Este registro es NULL en operaciones de delete o para triggers que se ejecutan por operación.

Registro OLD

Este registro contiene la información de los datos eliminados en la tabla en cuestión. Este registro es inyectado en las operaciones de delete sólo para triggers que se ejecutan a nivel de filas.

Este registro es NULL en operaciones insert o update o para triggers que se ejecutan por operación.

Creación de triggers

La sintaxis básica para crear triggers es la siguiente:

```
CREATE [ CONSTRAINT ] TRIGGER name { BEFORE | AFTER | INSTEAD OF } { event [ OR
... ] }
ON table
[ FROM referenced_table_name ]
[ NOT DEFERRABLE | [ DEFERRABLE ] { INITIALLY IMMEDIATE | INITIALLY
DEFERRED } ]
[ REFERENCING { { OLD | NEW } TABLE [ AS ] transition_relation_name } [ ... ] ]
[ FOR [ EACH ] { ROW | STATEMENT } ]
[ WHEN ( condition ) ]
EXECUTE PROCEDURE function_name ( arguments )
```

PostgreSQL admite especificar una condición booleana en la definición del trigger sobre la cual basa su decisión para ejecutar el trigger o no. Esta condición se especifica mediante la palabra ***WHEN***

Nota: si se definen varios triggers sobre la misma tabla y para el mismo evento, los mismos se ejecutarán en orden alfabético.

Nota II: puede existir recursividad en un trigger. Es responsabilidad del programador que no se creen loops infinitos.

Algunos ejemplos:

```
CREATE TRIGGER check_update
BEFORE UPDATE ON accounts
FOR EACH ROW
EXECUTE PROCEDURE check_account_update();
```

```
CREATE TRIGGER check_update
BEFORE UPDATE ON accounts
FOR EACH ROW
WHEN (OLD.balance IS DISTINCT FROM NEW.balance)
EXECUTE PROCEDURE check_account_update();
```

```
CREATE OR REPLACE FUNCTION before_insert_provincias() RETURNS trigger AS $$

BEGIN
       RAISE INFO 'ejecuta trigger';
       IF length(new.prv_nombre) < 3 THEN
          return null;
       END IF;
       return new;
END;

$$ LANGUAGE plpgsql;


CREATE TRIGGER before_insert_provincias
BEFORE INSERT ON provincias
FOR EACH ROW
EXECUTE PROCEDURE befor_insert_provincias();
```

En triggers before insert o before update, cuando realizamos validaciones y las mismas no se cumplen, podemos devolver NULL. Esto hace que el registro que va a tratar de insertar o

actualizar posee todos sus campos en NULL, lo que generará un error al tratar de realizar la operación.

Lectura recomendada

- PostgreSQL Triggers. Disponible en: https://www.postgresqltutorial.com/postgresql-triggers

Bibliografía

- Manejo de errores. Disponible en: https://www.postgresql.org/docs/14/plpgsql-errors-and-messages.html
- Códigos de error. Disponible en: https://www.postgresql.org/docs/14/errcodes-appendix.html
- PostgreSQL Triggers Documentation. Disponible en: https://www.postgresql.org/docs/current/sql-createtrigger.html
- PostgreSQL Trigger Procedures. Disponible en: https://www.postgresql.org/docs/10/plpgsql-trigger.html#PLPGSQL-DML-TRIGGER